Deep Learning with PyTorch

WEEK 2

Report for Week 2

1. Overview

Week 2 was the model-building phase. It introduced PyTorch fundamentals, moved from a feed-forward network to a convolutional image classifier, then studied how training choices affect performance through controlled experiments, and finished with a short technical report.

Task	Title	Level	Deliverable
2.1	Neural Network in PyTorch	Intermediate	PyTorch training notebook
2.2	CNN Image Classifier	Intermediate	Trained CNN model + accuracy report
2.3	Hyperparameter Experimentation	Intermediate	Experiment log with comparison table
2.4	Deep Learning Technical Report	Intermediate	1-2 page technical report (PDF)

2. Task-wise Details

2.1 Neural Network in PyTorch

INTERMEDIATE

Task Description

Implement a feed-forward neural network in PyTorch, covering tensors, datasets, dataloaders, and loss functions.

Objective

Learn the PyTorch training loop and the building blocks used by all deep learning models.

Approach and Methodology

- Created and manipulated tensors, and used autograd to compute gradients.
- Wrapped data in Dataset and DataLoader objects with batching and shuffling.
- Defined a feed-forward network with nn.Module using linear layers and ReLU activations.
- Selected a loss function (e.g. CrossEntropyLoss) and an optimizer (e.g. SGD or Adam).
- Wrote the training loop: forward pass, loss, zero_grad, backward, step; tracked loss and accuracy per epoch on train and validation sets.

Tools and Libraries

Python, PyTorch (torch, torch.nn, torch.utils.data), Matplotlib.

Outcome and Results

The network trained successfully with decreasing loss. *[Add: your actual result, e.g. dataset name, metric values, screenshots reference]*

Key Learnings

- The training loop is the same pattern in every PyTorch project.
- Monitoring validation loss reveals overfitting early.

DELIVERABLE PyTorch training notebook.

2.2 CNN Image Classifier

INTERMEDIATE

Task Description

Prepare an image dataset and train a convolutional neural network to classify images using PyTorch.

Objective

Build a CNN that learns visual features and classifies images accurately.

Approach and Methodology

- Loaded and preprocessed an image dataset: resizing, normalisation and tensor conversion; applied augmentation to reduce overfitting.
- Designed a CNN with convolution, ReLU, pooling and fully connected layers.
- Trained with a suitable loss and optimizer and saved the best checkpoint.
- Evaluated on a held-out test set using accuracy, a confusion matrix and sample predictions.
- Analysed misclassified images to understand model weaknesses.

Tools and Libraries

PyTorch, torchvision, Matplotlib.

Outcome and Results

The CNN was trained and evaluated and the model file saved. *[Add: your actual result, e.g. dataset name, metric values, screenshots reference]*

Key Learnings

- Convolutions share weights, making them efficient for images.
- Augmentation and validation tracking are key to generalisation.

DELIVERABLE Trained CNN model + accuracy report.

2.3 Hyperparameter Experimentation

INTERMEDIATE

Task Description

Run controlled experiments varying optimizers, learning rates, batch sizes, and epochs; compare outcomes.

Objective

Understand how training settings change convergence speed and final accuracy.

Approach and Methodology

- Fixed model, data and random seed, and changed one variable at a time for fair comparison.
- Compared optimizers (SGD, SGD with momentum, Adam), learning rates, batch sizes and epoch counts.
- Logged every run: configuration, final train/validation loss, accuracy and training time.
- Plotted loss and accuracy curves and built a comparison table to identify the best configuration.

Tools and Libraries

PyTorch, Pandas, Matplotlib.

Outcome and Results

A structured experiment log was created and the best configuration identified. *[Add: your actual result, e.g. dataset name, metric values, screenshots reference]*

Key Learnings

- Learning rate is usually the most sensitive hyperparameter.
- Controlled, logged experiments make results trustworthy and reproducible.

DELIVERABLE Experiment log with comparison table.

2.4 Deep Learning Technical Report

INTERMEDIATE

Task Description

Compare PyTorch model performance across experiments and summarize findings in a short written report.

Objective

Communicate technical results clearly and concisely.

Approach and Methodology

- Collected results from Tasks 2.1 to 2.3.
- Wrote a 1-2 page report covering objective, setup, results table, key plots and conclusions.
- Stated limitations and recommended next steps.

Tools and Libraries

Markdown/Word or LaTeX, exported to PDF.

Outcome and Results

The technical report was written and exported as PDF. *[Add: your actual result, e.g. dataset name, metric values, screenshots reference]*

Key Learnings

- Clear reporting is as important as the model itself.

DELIVERABLE 1-2 page technical report (PDF).

3. Summary of the Week

- Implemented neural networks from scratch in PyTorch.
- Trained and evaluated a CNN image classifier.
- Ran and documented controlled hyperparameter experiments.
- Summarised findings in a technical report.

4. Challenges and How They Were Handled

- Overfitting on the image dataset; reduced with augmentation and early stopping.
- Unstable training at high learning rates; resolved by lowering the learning rate and comparing optimizers.

5. Outlook

Week 3 broadens the toolkit with TensorFlow/Keras, a full CV or NLP project, prompt engineering and AI-assisted coding.